

A PRACTICAL ROADMAP

MLDLC

Machine Learning Development Life Cycle

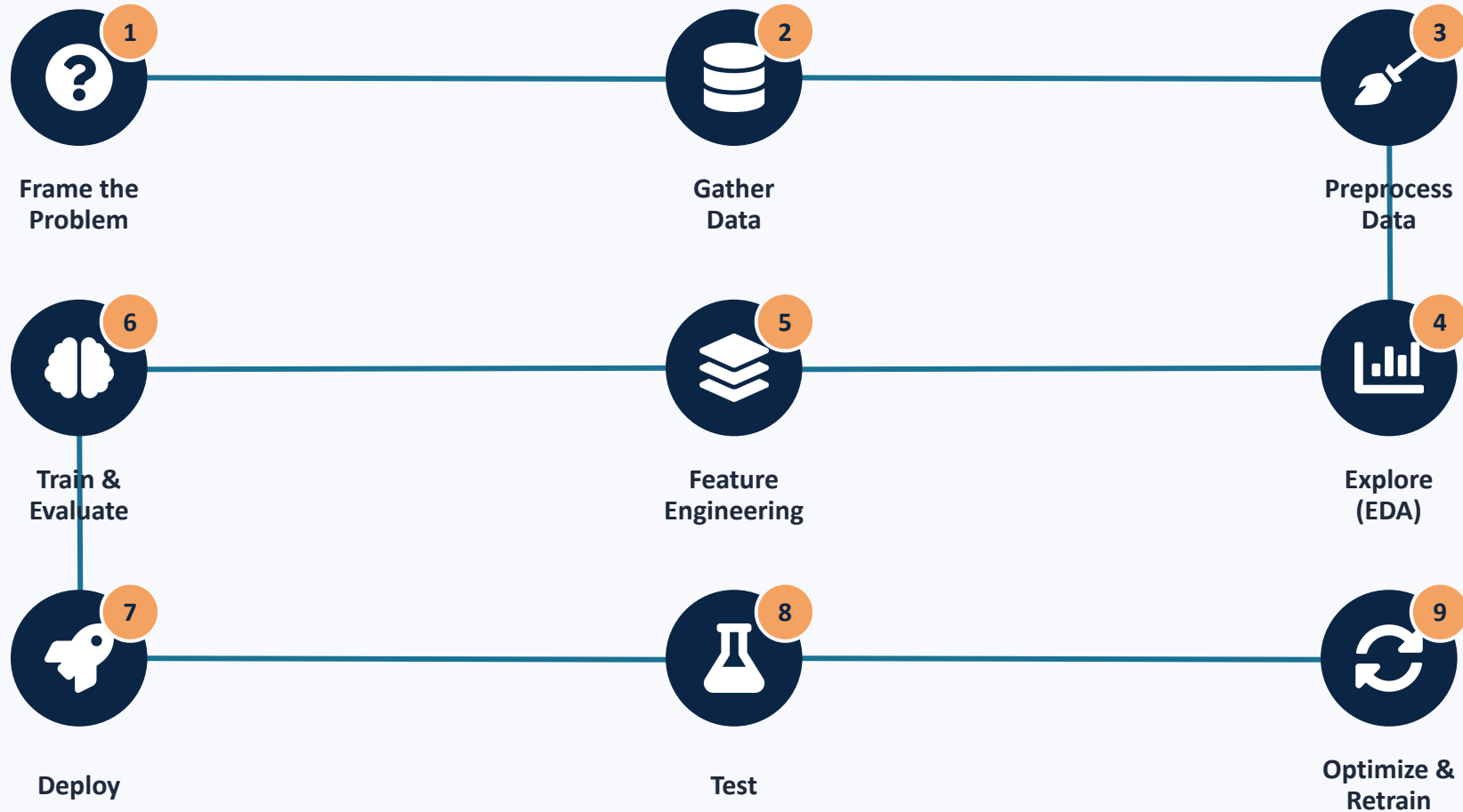
Turning a raw idea into a working, deployed ML product — in 9 steps



THE BIG PICTURE

Nine Steps, One Continuous Loop

Idea → Data → Model → Product → back to Idea



Frame the Problem



What's the problem?

Define the exact business or real-world problem you're trying to solve.



What does the output look like?

A price, a category, a recommendation? Know the shape of the answer first.



Online or offline?

Real-time predictions on the fly, or batch processing on stored data?



Where does data come from?

Files, APIs, websites, or internal databases — map the source early.



Supervised or unsupervised?

Do we have labeled answers to learn from, or just patterns to discover?



Who is the audience?

End-users, internal teams, or other systems — design the output for them.

Gather Data



Data can come from several different sources:



CSV Files

Flat files exported from spreadsheets or other tools.



APIs

Live data pulled programmatically from a service.



Web Scraping

Extracting data directly from websites.



Databases

Large-scale data warehouses storing huge clusters of records.



Good to know: Databases often hold huge data clusters that can't be modeled directly — they're first organized into a data warehouse, then the relevant slice is fetched for modeling.



Data Preprocessing

Raw data is messy — clean it up before it ever sees a model:



Remove Duplicates

Drop repeated rows that would bias the model.



Handle Missing Values

Fill in or remove gaps left by missing entries.



Detect & Remove Outliers

Find unusual values that distort patterns.



Scale the Values

Standardize so large and small numbers are on the same scale.

Standardization at a glance

Before (mixed scales)



After (standardized)





Exploratory Data Analysis

Get to know the data before modeling it:



Study input–output relationships

How do features relate to the thing we're predicting?



Visualize with charts

Plots reveal patterns numbers alone can hide.



Spot outliers visually

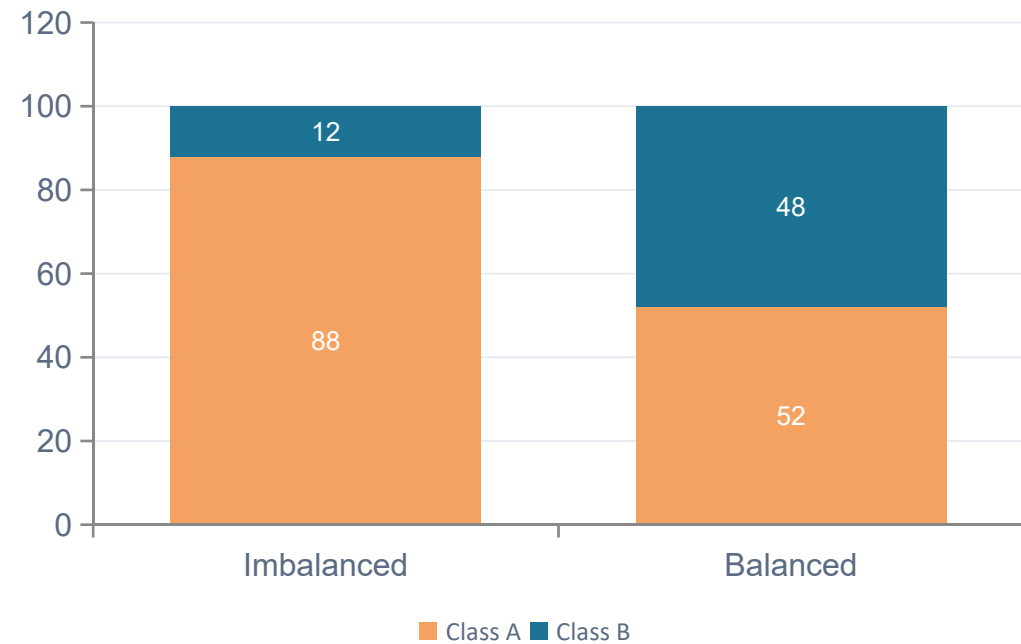
Box plots and scatter plots make anomalies obvious.



Check class balance

Confirm categories aren't wildly over- or under-represented.

Imbalanced vs. Balanced Classes





Feature Engineering & Selection

Shape the columns that actually go into the model:



Create new features

Combine or transform raw columns into more meaningful signals (e.g. turning a date into 'day of week').



Select relevant columns

Keep features that genuinely help predict the outcome.



Drop irrelevant ones

Remove columns that add noise instead of value.



Encode categories

Convert text/category columns into a numeric form models can use.



Model Training & Evaluation

Let the data choose the best algorithm:



Train multiple algorithms

Try several models on the same data — don't commit too early.



Evaluate with metrics

Accuracy, precision, recall, RMSE — pick metrics that fit the problem.



Tune hyperparameters

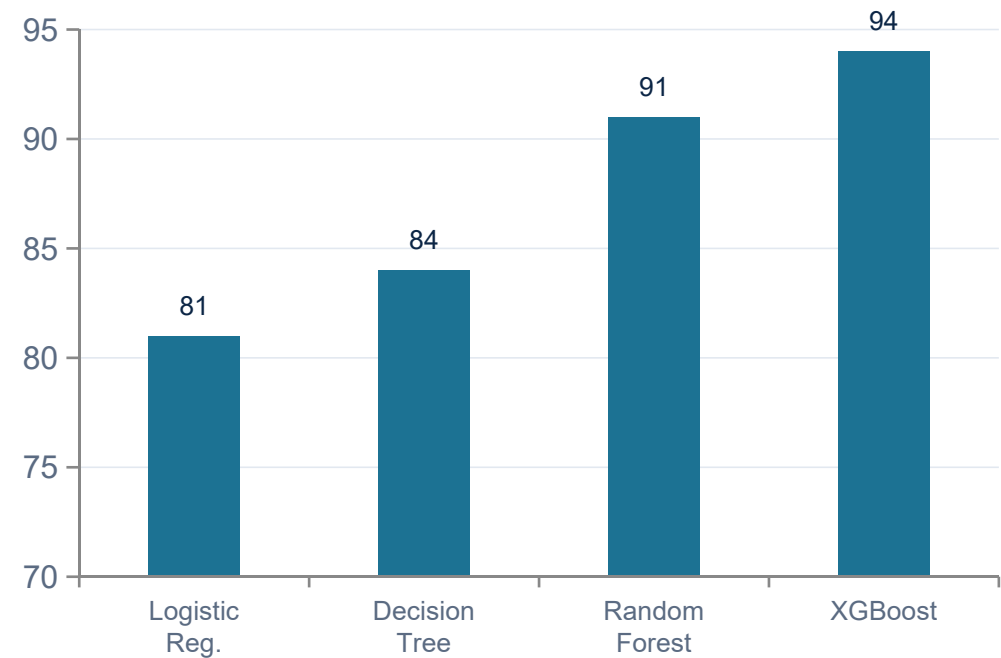
Adjust each model's settings to squeeze out better performance.



Choose the winner

Select the model that performs best and generalizes well.

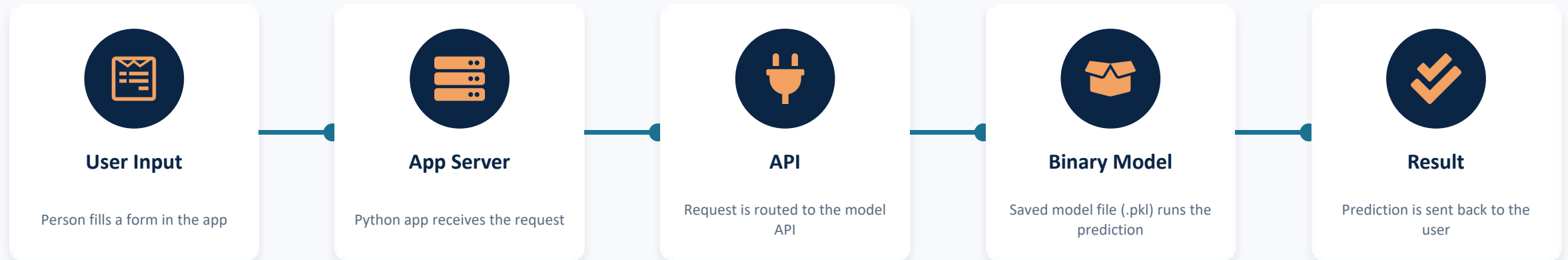
Comparing Algorithms by Accuracy





Model Deployment

The trained model is saved as a binary file, then wrapped behind an API so real users can reach it:



In short: the form's input travels through the server to the API, which loads the saved binary model, checks the input, and returns a prediction.

Testing



Before a full launch, validate the model in the real world:



Beta Testing

Release the model to a small group of trusted, real customers first — not the entire user base.



Confirms real-world accuracy

Lab metrics don't always match live behavior.



Surfaces usability issues

Trusted users flag confusing or broken flows early.



Limits the blast radius

Problems are caught before they reach everyone.

Optimize & Maintain



A shipped model still needs care — this is where MLOps takes over:



Backup Model & Data

Keep safe copies of both the model and the data behind it.



Handle the Load

Scale the system so it stays fast as traffic grows.



Decide Retrain Frequency

Define how often the model needs fresh data.



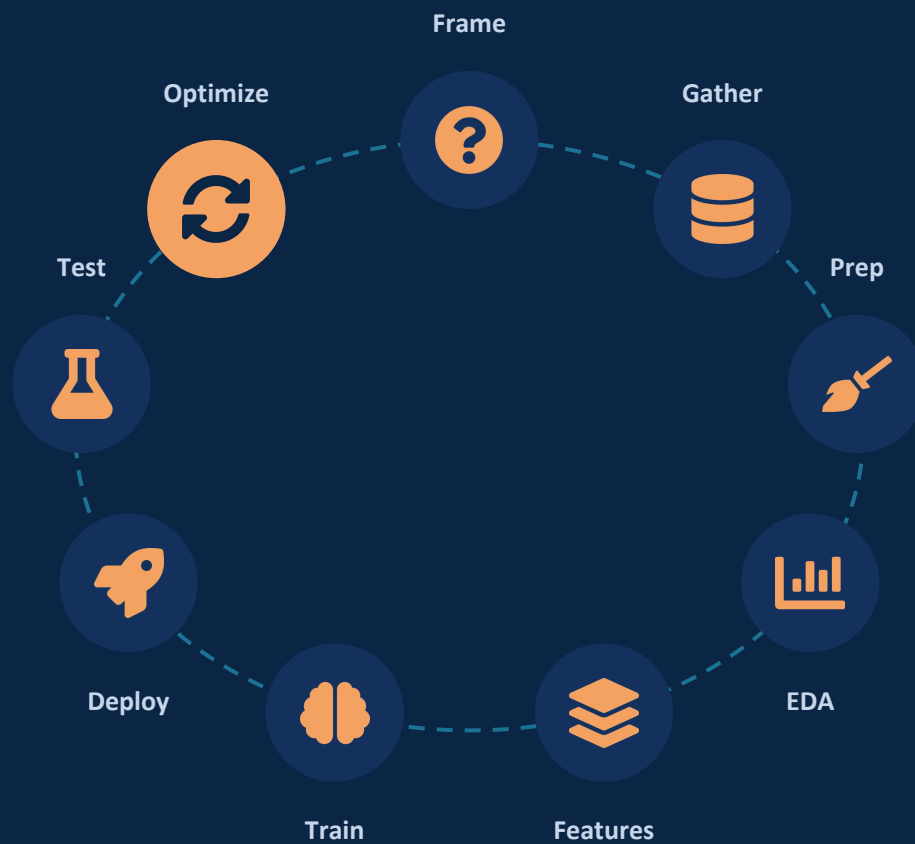
Automate Retraining

Build a pipeline so retraining happens on its own (MLOps).

REMEMBER

It's a Cycle, Not a Line

Step 9 feeds straight back into Step 1 — new data and feedback continuously reshape the problem.



Thank you